Trabajo Práctico Especial - Redes de Información

ITBA - 2C 2026

Tema: Gestión de Infraestructura por Código con Terraform

Integrantes:

Matias Rossi
Juan Pablo Birsa
Federico Viera

Índice

1. **Contexto y Problemática** 2
2. **Diseño de la Solución** 2
3. **SCOPE del POC y casos de uso** 3
4. **Diagrama de Arquitectura** 3
5. **Alternativas consideradas** 4
6. **Fuentes** 5

1. Contexto y Problemática

Toda aplicación necesita, además de su código, una infraestructura sobre la cual ejecutarse: redes con su direccionamiento, servidores, puntos de entrada, bases de datos y reglas de filtrado. Cuando esa infraestructura se arma a mano, con clics en una consola o con una secuencia de comandos, aparecen tres problemas conocidos: los entornos se vuelven irrepetibles, porque cada servidor termina siendo un ejemplar único configurado de memoria (el *snowflake server* de Fowler); no queda registro de qué se cambió, cuándo ni por qué; y la recuperación ante una falla depende de que alguien recuerde los pasos exactos. Estas limitaciones motivaron las herramientas de Infraestructura como Código (IaC), entre ellas Terraform.

The Store es una plataforma de comercio electrónico compuesta por cinco microservicios (UI, Catalog, Cart, Orders y Checkout) escritos en Java, Go y Node.js, cada uno con su propio motor de persistencia. La cátedra la provee junto con un script de bash (`local.sh`) que crea un clúster de Kubernetes local con kind, construye las imágenes de los cinco servicios, las carga en el clúster y aplica un manifiesto YAML con todos los objetos.

Ese script resuelve el arranque en una notebook, pero es imperativo: describe una secuencia de pasos y no el resultado deseado. No mantiene registro de lo que existe, por lo que sólo sabe crear todo o borrarlo todo; no detecta ni corrige las modificaciones hechas por fuera con `kubectl`; no permite revisar qué se va a modificar antes de aplicar un cambio; y no contempla parametrizar el entorno. En la nube el problema se agrava: habría que crear además redes, subredes, tablas de ruteo, clústeres, balanceadores, bases de datos y reglas de seguridad, y hacerlo a mano en la consola no es reproducible, auditable ni repetible entre entornos. Se propone entonces adoptar Terraform para declarar como código toda la infraestructura que The Store necesita en AWS, de manera versionable, auditable y reproducible.

2. Diseño de la Solución

Se utilizará Terraform (lenguaje HCL) con el provider de AWS para aprovisionar toda la infraestructura en la región `us-east-1`; con los providers `kubernetes` y `helm` para los objetos que corren dentro del clúster; con `docker` para construir las imágenes y publicarlas en el registro de contenedores de AWS (ECR); y con `random` para generar credenciales. La red se escribe recurso por recurso, porque es el objeto de estudio de la materia; para el clúster se usa el módulo oficial de la comunidad (`terraform-aws-modules/eks`), que resuelve los roles de IAM y los security groups que EKS exige. El entorno encendido cuesta cerca de 0,30 USD por hora, por lo que se crea en cada sesión de trabajo y se destruye al terminar: el código permite recrearlo idéntico cada vez.

La infraestructura gestionada por Terraform se organiza en los siguientes módulos:

- **Networking:** VPC 10.0.0.0/16 en dos zonas de disponibilidad (`us-east-1a` y `us-east-1b`), con una subred pública (10.0.0.0/24 y 10.0.1.0/24) y una privada (10.0.16.0/20 y 10.0.32.0/20) en cada zona, Internet Gateway, un NAT Gateway y las tablas de ruteo. Las públicas alojan el balanceador y el NAT Gateway; las privadas, los nodos, los pods y las bases de datos, sin direcciones públicas.
- **Cluster:** clúster EKS (Kubernetes 1.34) con un grupo administrado de dos nodos `t3.medium` (Amazon Linux 2023) en las subredes privadas, el bloque de servicios 172.20.0.0/16, el plugin de red VPC CNI con NetworkPolicies habilitadas y el Ingress Controller (nginx) publicado mediante un Network Load Balancer en las subredes públicas, único punto de entrada a la aplicación.
- **Registry:** un repositorio ECR por microservicio. Terraform construye las cinco imágenes a partir del código fuente provisto por la cátedra y las publica con el tag que indica la variable `app_version`.
- **Databases:** los servicios administrados de datos que la aplicación espera: RDS MySQL para Catalog, RDS PostgreSQL para Orders, ElastiCache Redis para Checkout, una tabla DynamoDB para Cart y una cola SQS para los eventos de Orders. RDS y ElastiCache viven en las subredes privadas con un security group que sólo acepta conexiones desde los nodos del clúster; DynamoDB y SQS se protegen con permisos IAM asignados al rol de los nodos. Las contraseñas las genera Terraform: no hay credenciales escritas en el código.
- **Application:** los cinco microservicios, cada uno con su Deployment (que mantiene N copias del proceso) y su Service (nombre DNS interno e IP virtual que balancea entre las copias); el Ingress que publica la tienda; y las NetworkPolicies que segmentan el tráfico dentro del clúster siguiendo el principio de mínimo privilegio.

El código se organiza en un directorio `terraform/` con la configuración raíz (`main.tf`, que orquesta los módulos; `variables.tf` y `terraform.tfvars`, con los parámetros del entorno: región, bloques de red, cantidad de nodos, réplicas y versión; `outputs.tf`; `providers.tf`; y `backend.tf`), un subdirectorio por módulo en `modules/` y un

`bootstrap/` que crea, una única vez, el bucket S3 donde se guarda el estado de Terraform con versionado y bloqueo nativo (`use_lockfile`), para que los tres integrantes trabajen sobre el mismo entorno sin pisarse y quede el historial de cada cambio.

3. SCOPE del POC y casos de uso

Scope del proyecto:

- Provisionar la VPC 10.0.0.0/16 con sus cuatro subredes en dos zonas de disponibilidad, Internet Gateway, NAT Gateway y tablas de ruteo.
- Desplegar un clúster EKS (Kubernetes 1.34) con dos nodos `t3.medium` en las subredes privadas, el Ingress Controller y el Network Load Balancer que publica la tienda.
- Construir las imágenes de los cinco microservicios a partir del código fuente provisto y publicarlas en ECR.
- Aprovisionar RDS MySQL, RDS PostgreSQL, ElastiCache Redis, DynamoDB y SQS, y conectar los cinco microservicios a ellos.
- Aplicar security groups y NetworkPolicies según el principio de mínimo privilegio, y guardar el estado de Terraform en S3 con versionado y bloqueo.
- Demostrar el plan de ejecución (`terraform plan`) previo a cada cambio, la creación (`terraform apply`) y la destrucción (`terraform destroy`) completas del entorno, y los casos de uso descritos a continuación.

Fuera de alcance: dominio propio y certificados TLS, integración continua, observabilidad, gestión centralizada de logs, service mesh, autoescalado automático y alta disponibilidad de las bases de datos (Multi-AZ), por corresponder a los temas de otros grupos, por costo o por no ser necesarios para demostrar el objetivo del trabajo.

Casos de uso:

1. **Creación y destrucción completa del entorno de forma reproducible.** Toda la infraestructura (red, clúster, imágenes, bases de datos y aplicación) se crea desde cero con `terraform apply` y se elimina por completo con `terraform destroy`, sin dejar recursos huérfanos en la cuenta. Como complemento se demuestra la detección de *drift*: se modifica un objeto a mano (la cantidad de réplicas de un servicio con `kubectl`), `terraform plan` muestra la diferencia entre lo declarado y lo real, y `terraform apply` la corrige.

2. **Escalamiento ante un pico de demanda.** Para absorber un volumen de tráfico mayor al habitual se modifica `terraform.tfvars` aumentando las réplicas de los microservicios más solicitados (la UI de 2 a 4) y la cantidad de nodos worker del clúster (de 2 a 3). `terraform plan` muestra el alcance del cambio y `terraform apply` lo ejecuta: el grupo de nodos administrado lanza una nueva instancia EC2 en las subredes privadas y Kubernetes reparte las nuevas copias. Un cambio de configuración en un único archivo escala tanto la infraestructura como la aplicación de forma controlada.

3. **Actualización de versión y rollback.** Se descubre un problema en la versión desplegada. Se modifica la variable que define la versión de las imágenes y se aplica el cambio: Terraform reconstruye las imágenes, las publica en ECR y Kubernetes ejecuta un *rolling update* de los pods afectados sin interrumpir el servicio. Si la nueva versión introduce fallas, se revierte la variable al valor anterior y se vuelve a aplicar.

4. **Respuesta a un incidente de seguridad.** Se detecta que uno de los microservicios (por ejemplo, Checkout) fue comprometido. Como medida de contención se lo declara en cuarentena en `terraform.tfvars`. `terraform plan` muestra que el único cambio es la capa a la que pertenece ese servicio y, al aplicarlo, las reglas de segmentación dejan de permitirle cualquier comunicación: no recibe pedidos de la UI ni puede conectarse a otros servicios, a las bases de datos ni a Internet. El resto de la tienda sigue operando (catálogo y carrito) y sólo la función del servicio aislado queda deshabilitada hasta resolver el incidente. Levantar la cuarentena es vaciar la lista y volver a aplicar.

4. Diagrama de Arquitectura

Todo el entorno se despliega en la región `us-east-1` dentro de una VPC con el bloque **10.0.0.0/16**, repartida en dos zonas de disponibilidad. En cada zona hay una subred pública (**10.0.0.0/24** y **10.0.1.0/24**) y una privada

(**10.0.16.0/20** y **10.0.32.0/20**). Las privadas son más grandes porque alojan los nodos, sus pods y las bases de datos: en EKS los pods toman direcciones de la propia subred (plugin VPC CNI), no de un bloque aparte. Las públicas rutean por defecto al Internet Gateway; las privadas, a un NAT Gateway ubicado en la subred pública de la primera zona, que enmascara el tráfico saliente con una única IP elástica (SNAT). Dentro de la VPC todas las subredes se alcanzan por la ruta local. Los Services de Kubernetes usan el bloque virtual **172.20.0.0/16**, que sólo existe en las reglas de DNAT que `kube-proxy` mantiene en cada nodo; CoreDNS responde en 172.20.0.10 y el resolutor de la VPC (10.0.0.2) resuelve los endpoints de los servicios de AWS.

Los nodos son instancias EC2 `t3.medium` con Amazon Linux 2023 (kubelet y containerd) en las subredes privadas, sin IP pública y con un security group propio. El plano de control de EKS lo opera AWS fuera de la VPC y se alcanza por HTTPS (443) con autenticación de IAM, tanto desde los nodos como desde nuestras estaciones de trabajo. Las imágenes de los microservicios están basadas en Amazon Linux 2023.

Un pedido del navegador entra por el Internet Gateway al Network Load Balancer (balanceo de capa 4, puerto 80), que lo reenvía al puerto de nodo donde escucha el Ingress Controller; éste lo enruta (capa 7) al Service de la UI, cuya IP virtual se traduce por DNAT a uno de los pods de UI en el puerto 8080. La UI consulta por HTTP a los cuatro backends mediante sus nombres DNS internos, y éstos a sus bases con el protocolo de cada motor: MySQL (3306) y PostgreSQL (5432) en RDS y RESP (6379) en ElastiCache, protegidos por un security group que sólo admite el security group de los nodos; y HTTPS (443) hacia DynamoDB y SQS a través del NAT Gateway, con permisos IAM. Dentro del clúster, las NetworkPolicies aplican mínimo privilegio: por defecto ningún pod acepta conexiones entrantes; la UI sólo acepta tráfico del Ingress Controller; los backends sólo de la UI y de otros backends; y un servicio en cuarentena no puede recibir ni iniciar conexiones.

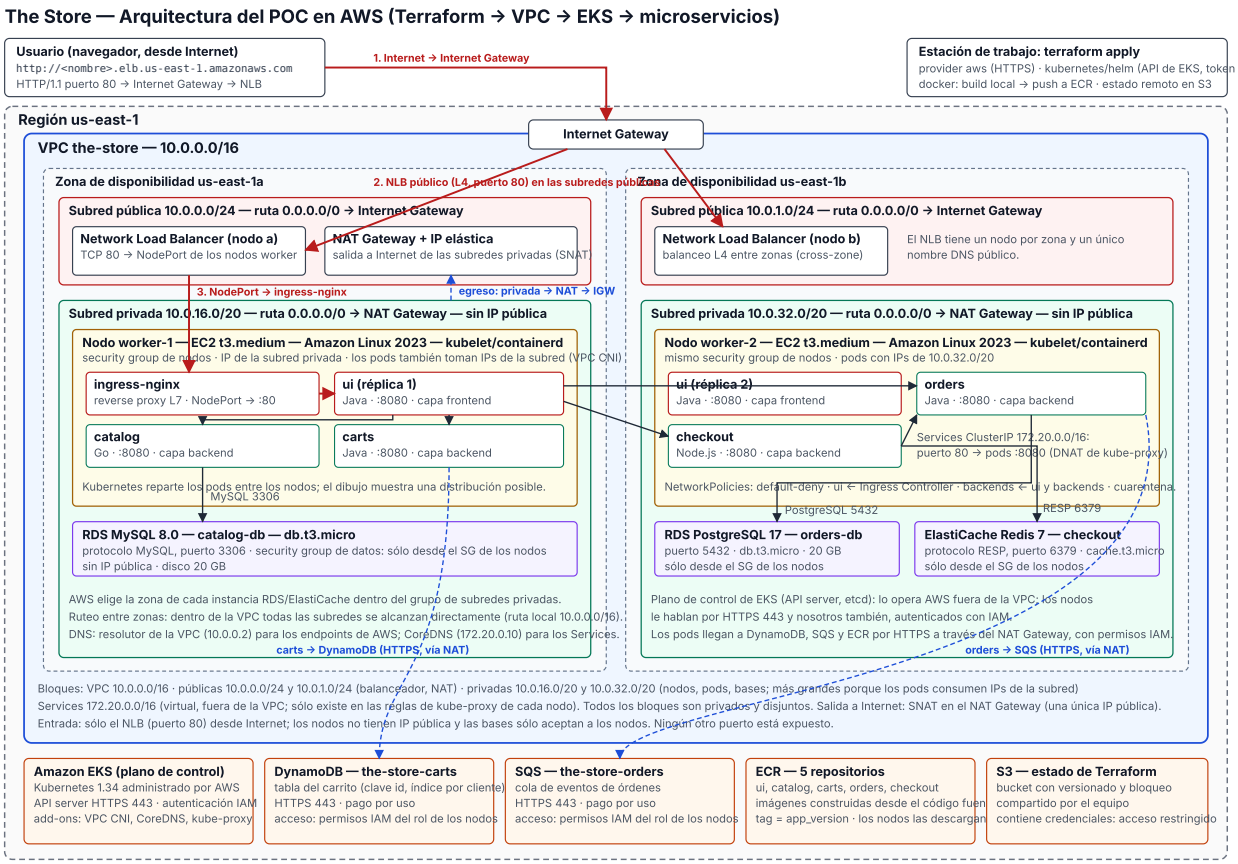


Figura 1: Arquitectura del POC en AWS. Rojo: tráfico del usuario; negro: tráfico interno; azul punteado: egreso hacia servicios de AWS por el NAT Gateway.

5. Alternativas consideradas

Como alternativa a **Terraform** se consideró **Pulumi**, que declara infraestructura con lenguajes de propósito general (TypeScript, Python, Go). Se optó por Terraform porque HCL es un lenguaje declarativo específico para infraestructura, más legible y con un plan de cambios directo de revisar, y por la madurez de su ecosistema de providers. **OpenTofu**, su bifurcación de código abierto, es compatible con el código escrito; se mantuvo Terraform por ser la herramienta nombrada en el tema asignado.

Respecto de la plataforma, el primer prototipo se hizo sobre un **clúster local con kind**, la herramienta del script de la cátedra: es gratuito y sirvió para validar el módulo de aplicación. Se descartó como solución final porque kind no permite agregar nodos a un clúster existente, no ofrece balanceadores ni bases de datos administradas, y su red es una simulación dentro de una única máquina. EKS permite demostrar el escalamiento real de nodos, los servicios administrados y una red con subredes, ruteo, NAT y security groups verdaderos, a cambio de un costo por hora que se controla destruyendo el entorno tras cada sesión.

Para la red se evaluó el módulo comunitario `terraform-aws-modules/vpc`; se prefirió escribir la VPC recurso por recurso para que las subredes, las tablas de ruteo y el NAT Gateway queden explícitos en el código y puedan explicarse en la presentación. Para el clúster, en cambio, se usa el módulo comunitario de EKS, porque los roles de IAM y security groups que requiere no aportan al análisis de red y es fácil equivocarse al escribirlos a mano.

Para el punto de entrada se consideró el **AWS Load Balancer Controller** con un Application Load Balancer. Se eligió el Ingress Controller nginx con un Network Load Balancer porque no requiere instalar un controlador adicional ni permisos IAM extra, y muestra con claridad la diferencia entre el balanceo de capa 4 (NLB) y el de capa 7 (nginx). Para los permisos de los pods sobre DynamoDB y SQS se consideró **IAM Roles for Service Accounts** (un rol por microservicio); se optó por asignar los permisos al rol de los nodos por simplicidad del POC, dejando documentada la alternativa como mejora.

Finalmente se consideró resolver el despliegue sólo con **Helm** o con **Ansible**. Helm alcanza únicamente a los objetos de Kubernetes: no crea la VPC, el clúster ni las bases de datos. Ansible está orientado a configurar servidores y no mantiene un estado contra el cual comparar la realidad, que es justamente lo que se quiere demostrar. Ambos son complementarios: aquí Helm se invoca desde Terraform para instalar el Ingress Controller.

6. Fuentes

[1] HashiCorp. *What is Terraform?* `https://developer.hashicorp.com/terraform/intro`

[2] Fowler, M. (2012). *SnowflakeServer*. `https://martinfowler.com/bliki/SnowflakeServer.html`

[3] Morris, K. (2020). *Infrastructure as Code: Dynamic Systems for the Cloud Age* (2ª ed.). O'Reilly Media.

[4] Amazon Web Services. *Amazon EKS User Guide: Networking* y *Amazon VPC User Guide*. `https://docs.aws.amazon.com/eks/latest/userguide/eks-networking.html`

[5] terraform-aws-modules. *AWS EKS Terraform module*. `https://github.com/terraform-aws-modules/terraform-aws-eks`

[6] Kubernetes. *Services, Load Balancing and Networking* y *Network Policies*. `https://kubernetes.io/docs/concepts/services-networking/`

[7] AWS Containers. *Retail Store Sample Application*. `https://github.com/aws-containers/retail-store-sample-app`